Route Planning on Road and Route Graphs

Contents

1. Overview . . . 2
2. Why route planning is its own level . . . 4
3. Roads as a graph . . . 6
   - 3.1 Intersections, road segments and one-way streets . . . 6
   - 3.2 Lane graphs . . . 8
4. Travel time as the edge cost . . . 10
5. A heuristic in seconds: the straight line at the top speed . . . 12
   - 5.1 The bound and why it holds . . . 12
   - 5.2 Why the bound is loose . . . 13
6. Landmarks: lower bounds from stored travel times . . . 16
   - 6.1 The triangle inequality on travel times . . . 16
   - 6.2 The search with a landmark . . . 18
   - 6.3 Many landmarks on a real map . . . 20
7. Contraction hierarchies: shortcuts over unimportant roads . . . 21
   - 7.1 Why skip small roads . . . 21
   - 7.2 Contracting a vertex: shortcuts . . . 22
   - 7.3 The query: two searches that only climb . . . 23
   - 7.4 CH-Potentials: the hierarchy as a heuristic . . . 25
8. Summary . . . 26
9. Sources . . . 27
10. Key terms . . . 28

Where this fits:

- **Builds on:** Graphs and state spaces; Dijkstra's algorithm; A* search; Admissible heuristic; Backward and bidirectional search; Metric.
- **Compare with:** Heuristic.

1. Overview

Key point: Across a city, a robot or a car plans on a graph of roads whose edges cost travel time, and the search loop is A* as before. What decides how much of the map the search looks at is the heuristic: on a Chicago street graph of 1628 intersections, one trip takes 1526 intersections off the open list with no heuristic, 1229 with the straight line at the top speed and 108 with 16 landmarks, while a contraction hierarchy, which adds shortcuts before any search, takes 159.

The robot can already plan its way to the door of its room. A* (G-2577) on the room's 0.5 m grid takes 9 cells off its open list (G-2547), the cells reached but not yet expanded, where Dijkstra's algorithm takes 54, because a **heuristic** (G-2570), a cheap guess of the distance left, points it at the door.

Past the door, the trips get longer. A delivery robot, or a car, must cross a city: from a street corner in the south-west of the map of Figure 1 to one near the lake shore in the north-east, 4.6 km apart in a straight line. Three problems arise:

- **Size.** The map covers 4.9 km by 5.0 km; cut into 0.5 m cells like the room, it would be 96 million cells, against the room's 112.
- **Roads.** A car may not drive across a block. The car keeps to the streets, in the allowed direction, and in the right lane.
- **Time.** A trip is judged by how long it takes, not how long it is: the fastest route is 7047 m long and takes 492.8 s, while the shortest, 6717 m, takes 573.1 s (notebook).

Roads fix the size: the map shrinks to its 1628 intersections and the streets between them. The search effort is what is left. Figure 1 runs the same trip four ways, 40 intersections per frame in every panel: watch the red dots, the intersections taken off the open list, fill most of the map in the two top panels and stay close to the route in the two bottom ones.

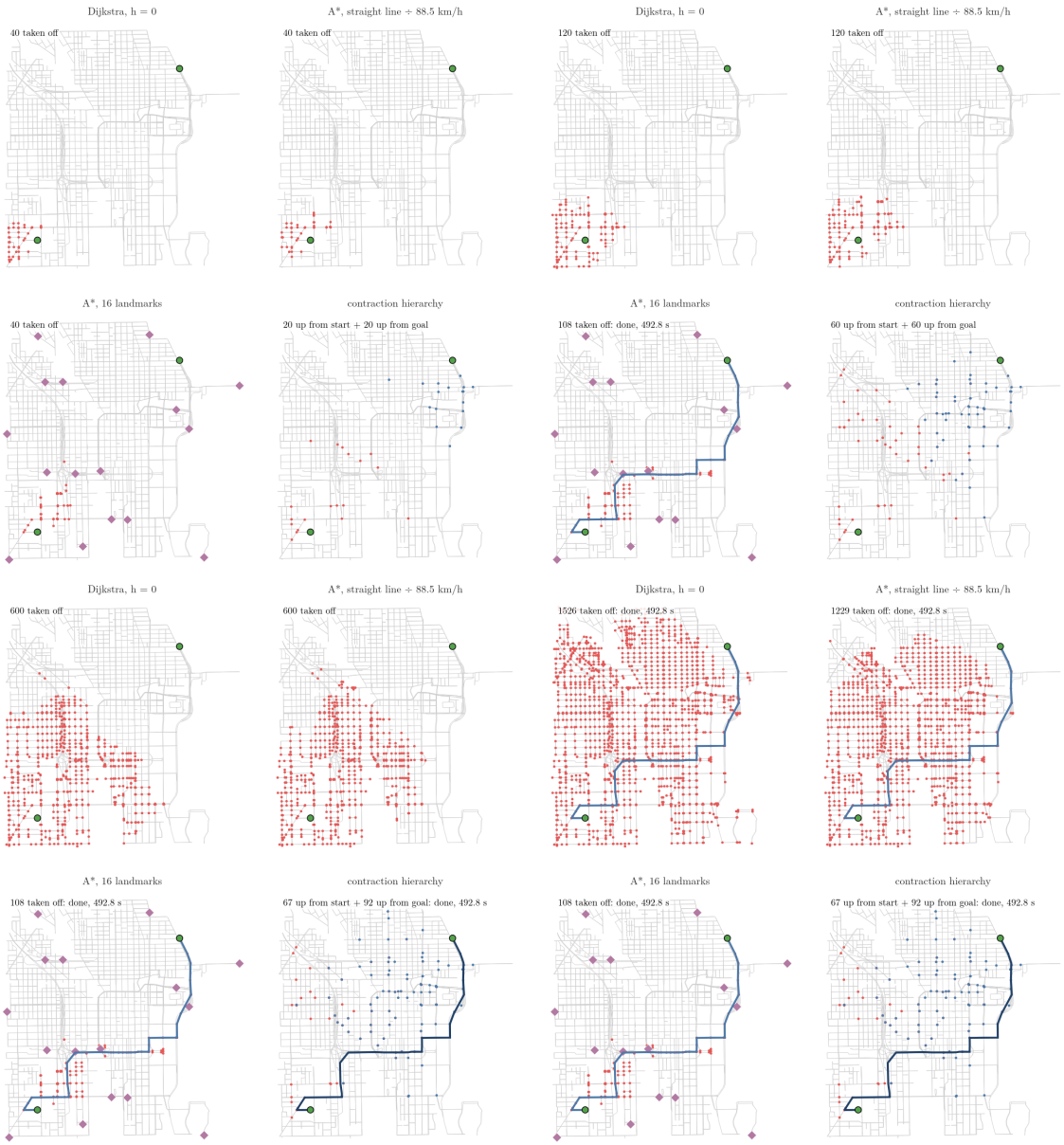


Figure 1: The same trip on the Chicago street graph, from the green dot in the south-west to the green dot in the north-east, searched four ways with travel-time costs. 40 intersections are taken off the open list per frame in every panel; a panel that has finished holds its last frame. Top left: Dijkstra's algorithm (heuristic 0), 1526 intersections. Top right: A* with the straight line to the goal at the top speed limit, 88.5 km/h, 1229. Bottom left: A* with 16 landmarks (purple diamonds), 108. Bottom right: a contraction hierarchy, a search up from the start (red) and one up from the goal (blue), 67 + 92 = 159. All four return the same 492.8 s route (blue line)

All four searches return the same route. In the first three the search loop is the same and only the heuristic, the guess of the time left, changes, and with it the work: from 1526 intersections to 108. The fourth adds shortcuts to the graph before any search and takes 159.

This Note builds the four, on a small town first and then on the Chicago map:

- why route planning is a level of its own, above the planners of the room (Section 2);
- the road network as a graph of intersections and one-way edges, and of lanes (Section 3);
- travel time as the cost of a road (Section 4);
- the first heuristic in seconds, the straight line at the top speed, and why it is loose (Section 5);
- a tight heuristic from stored travel times to a few landmarks (Section 6);
- contraction hierarchies, which add shortcuts so that a search only climbs to more important roads, and a heuristic built from them (Section 7).

2. Why route planning is its own level

> **Key point:** A car's drive cannot be planned as one problem from the destination down to the steering angle. Planning is split into levels; the top one, route planning, picks the roads and lanes on a map of the road network and leaves every detail of the motion to the levels below.

The first problem is size. The room's planners worked on every free cell and every heading; across the city, the same detail means 96 million cells, other cars, traffic lights and lane markings, for a drive of 8 minutes.

The idea. A taxi driver does not plan every turn of the steering wheel before leaving. First the driver decides the roads: west on this street, onto the expressway, off at the river. Only while driving does the driver decide when to change lane, whether to stop at a light, and how hard to brake. Each decision uses the one above it and adds detail.

Planning for self-driving cars is split the same way, because driving is hard to model as a single optimisation problem (Tübingen L12 slides; Paden et al. 2016). Figure 2 shows the levels.

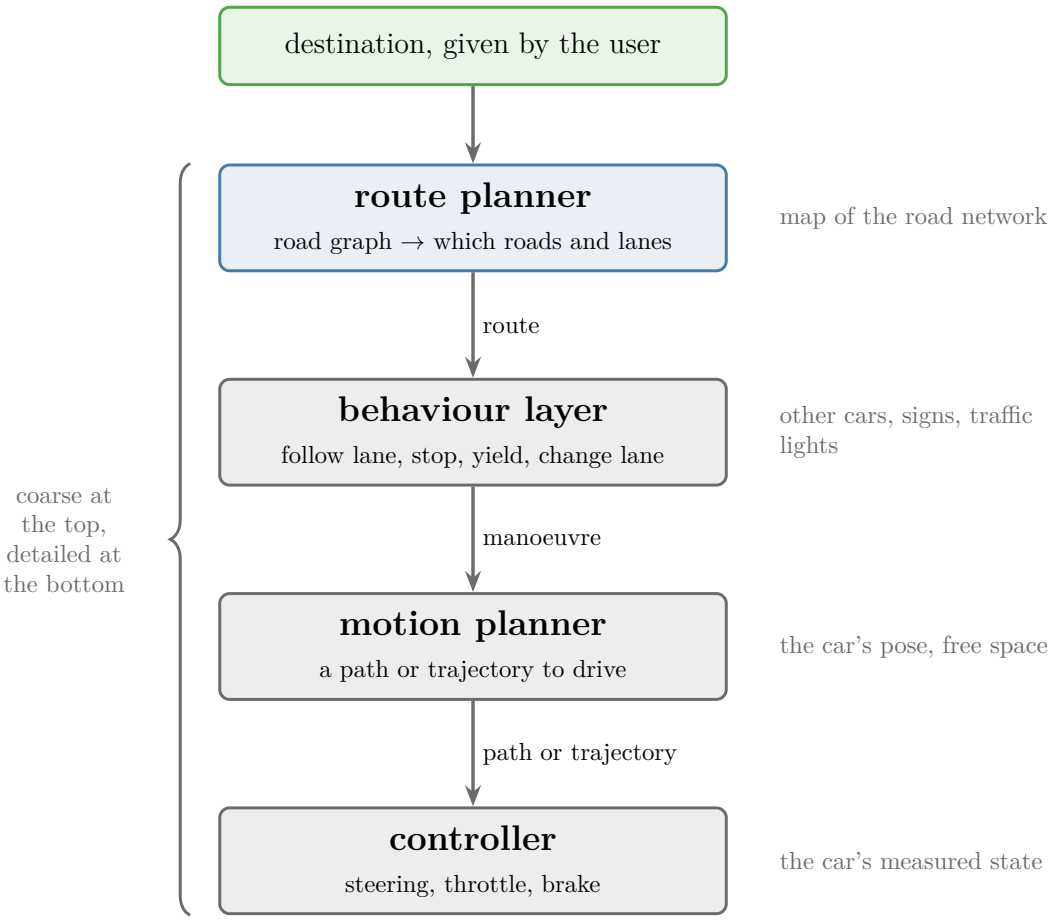


Figure 2: The planning levels of a self-driving car, from the destination at the top to the steering, throttle and brake at the bottom. The route planner turns the destination and a map of the road network into a route; the behaviour layer picks a manoeuvre (follow the lane, stop, yield, change lane) from what the car sees; the motion planner turns the manoeuvre into a path or trajectory; the controller follows it. Each level works coarser than the one below it

The standard term. Finding the best way from where the vehicle is to the destination on a map of the road network, by time or by distance, is **route planning** (G-2870), also called mission planning (Tübingen L12 slides). Route planning is the top level of Figure 2:

1. the route planner chooses the roads and lanes;
2. the behaviour layer chooses manoeuvres, such as stopping at a stop line or changing lane;
3. the motion planner finds a path or a trajectory that carries out the manoeuvre, with the planners of this chapter;
4. the controller turns it into steering, throttle and brake.

Earlier levels work with coarser pictures of the world (Tübingen L12 slides). A route planner never sees a parked car; the motion planner never sees the whole city.

Robots do the same. A delivery robot in a building can plan on a fixed graph of waypoints, such as corridor junctions and doors, the room's door among them, and leave the last few metres to a free-space planner such as A* on the room's grid (Nav2 Route Server). A fixed graph of points a robot may travel between, made by hand or by a program from the map, is a **route graph** (G-2872) (Nav2 Route Server). On route graphs, Dijkstra's algorithm alone is fast enough: 0.232 ms on average on a graph of 10,000 nodes and 44 ms on a million (Nav2 Route Server). A road graph of Western Europe has 18 million intersections (Geisberger et al. 2008), and there the search effort matters.

The route planner needs the road network as a graph; the next section builds one.

3. Roads as a graph

> **Key point:** Intersections are the vertices and road segments the edges of a road graph, and a one-way street is an edge that goes one way only, so road graphs are directed. When the route must say which lane to be in, each lane segment becomes a vertex of its own, joined to the next segment and to the lanes beside it.

Section 2 left the route planner a map of the road network; this section turns it into the **graph** (G-2540), points joined by lines, that a search needs. A road map is the next step up from the room, with roads instead of free floor, so we leave the room for a town.

3.1 Intersections, road segments and one-way streets

A small town. We follow every step by hand on a town of 11 intersections, small enough to check each number (Figure 3). A driver at S wants to reach T. The town has three kinds of road:

- a **bypass** north of the town, from N to M and on to L, at 80 km/h;
- the **main road**, which joins S to the town and the town to T and to the bypass, at 50 km/h;
- the **old town**, B, C, E and F, at 30 km/h; the street from F to C is one-way, northbound.

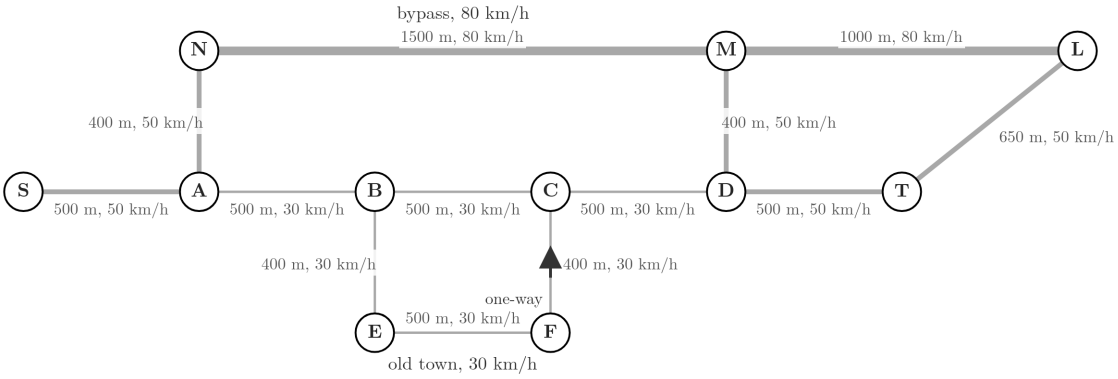


Figure 3: The town graph: 11 intersections (circles) joined by 13 streets, each labelled with its length and speed limit; a wider line is a faster road. The bypass, at 80 km/h, runs from N to M and on to L; the old town, at 30 km/h, has the streets between B, C, E and F; the street from F to C is one-way (arrow). The driver starts at S and wants to reach T

The standard terms. Each intersection is a **vertex** (G-2541) and each road segment between two intersections an **edge** (G-2542). A graph of the road network built this way is a **road graph** (G-2871) (Tübingen L12 slides; Lanelet2 routing). Every position is measured in one map frame: metres east and north of the map's south-west corner.

The room's moves went both ways; the street from F to C does not. A graph in which each edge has a direction, from one vertex to another, is a **directed graph** (G-2873) (Tübingen L12 slides). A two-way street becomes two edges, one each way, so the town's 13 streets are 25 edges: 12 two-way streets give 24, and the one-way street gives 1 (notebook). Road networks are directed graphs (Tübingen L12 slides).

A real road graph. Figure 4 is the graph we use for the Chicago trip, from OpenStreetMap: the streets a car may drive within 2.5 km of the city centre, made into a graph by the OSMnx library (Boeing 2025). The saved graph has 1670 intersections and 3946 streets; we keep the 1628 intersections from which every other one can be reached, joined by 3857 edges, and 1409 of those edges are one-way streets (notebook). The opening's 96 million cells shrink to 1628 vertices.

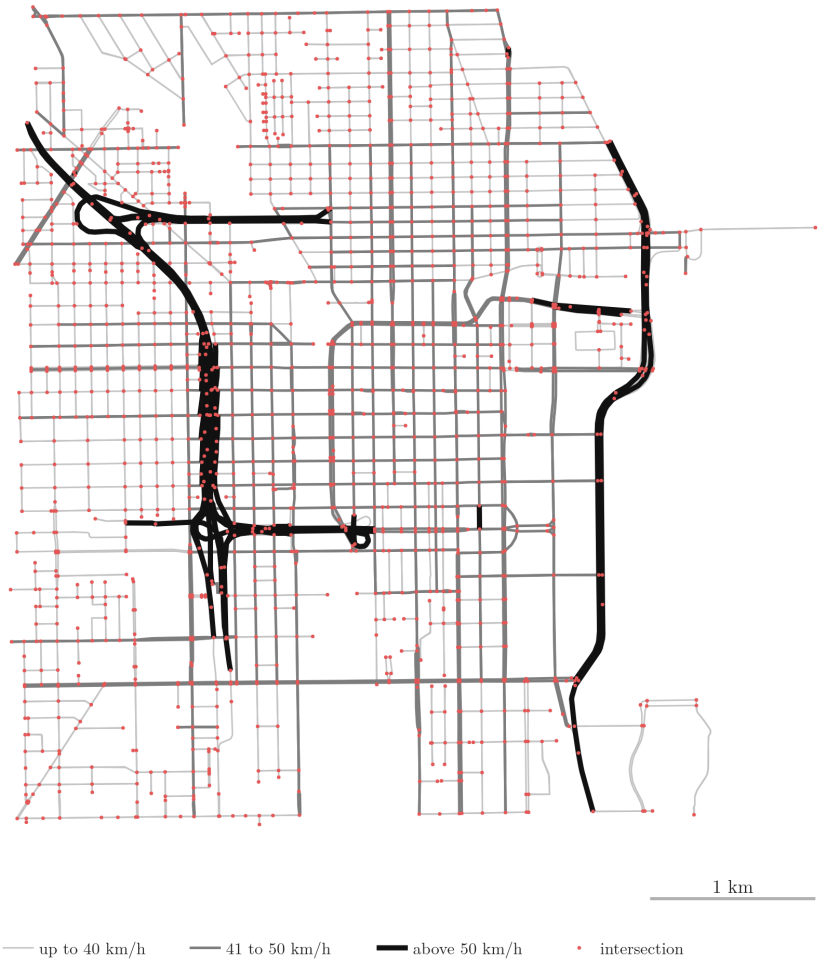


Figure 4: The Chicago street graph: every street a car may drive within 2.5 km of the city centre (OpenStreetMap), drawn wider and darker the higher its speed limit; red dots are the 1628 intersections. The black lines, above 50 km/h, are the expressways; most streets are 33 to 44 km/h. The map is 4.9 km wide and 5.0 km tall

Larger maps are the same kind of graph, only bigger: the road graph of New York City has 54,837 vertices and 140,497 edges (Tübingen L12 slides).

3.2 Lane graphs

A road graph has one vertex per intersection, so it cannot say which lane the car should be in. The lane matters before a turn. In Figure 5 a car in the right lane that must turn left at the junction has to change to the left lane first, and it cannot do so at the last metre.

The idea. Cut every lane into short pieces and make each piece a vertex. Join a piece to the piece that follows it in the same lane, and to the pieces beside it that the car may change into.

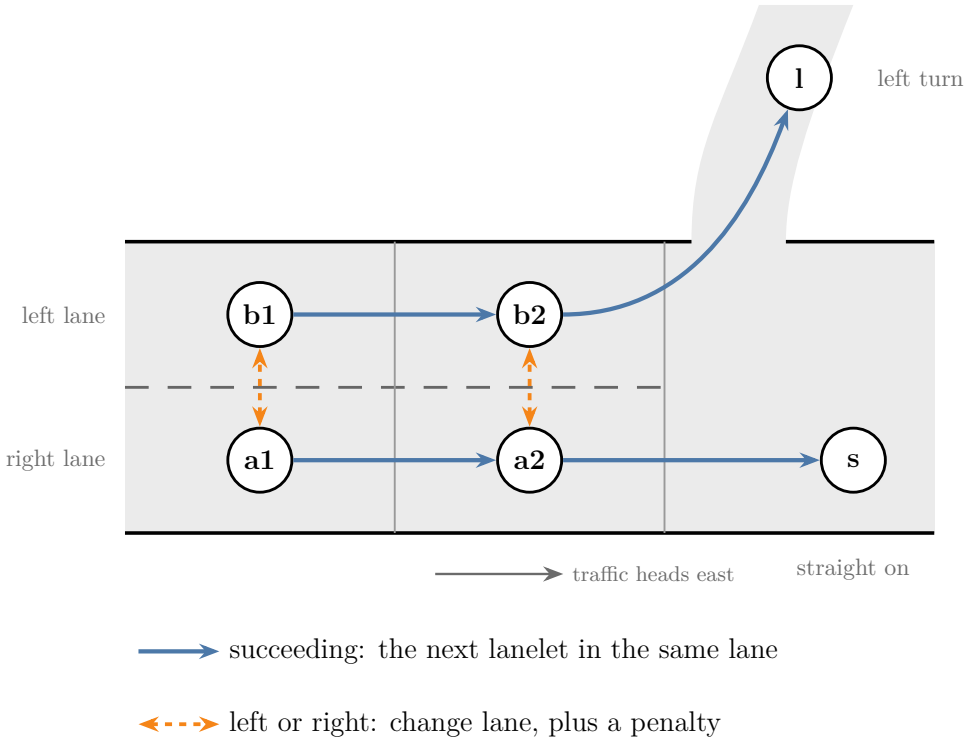


Figure 5: A lane graph on a two-lane road heading east. Each lane is cut into pieces, the vertices a1 and a2 (right lane), b1 and b2 (left lane), s (straight on after the junction) and l (the left turn). Blue arrows join a piece to the next piece in the same lane; orange dashed arrows join pieces a car may change between, at a penalty. To turn left from a1, the route must cross to the left lane: a1, b1, b2, l or a1, a2, b2, l

The standard terms. One piece of one lane, with its left and right borders, is a **lanelet** (G-2875) (Lanelet2 routing). A graph whose vertices are lanelets is a **lane graph** (G-2874). Its edges are of two kinds (Lanelet2 routing):

- **succeeding:** from a lanelet to the next one in the same lane, such as a1 to a2;
- **left and right:** to a neighbouring lanelet the car may change into, such as a1 to b1; a lane change adds a penalty to the cost, so a route changes lane only when the change lowers its cost.

Lanes beside each other that may not be changed between, such as two lanes with a solid line between them, get no lane-change edge. The graph is also built for one kind of road user: a bicycle's lane graph includes cycle lanes that a car's does not (Lanelet2 routing).

To turn left from a1 in Figure 5, a route must include a lane change: a1, b1, b2, l or a1, a2, b2, l. A route planner that runs on a lane graph returns a list of lane IDs; the Autoware software for self-driving cars groups them into sections of road, each with the lanes the car may change between (Autoware mission planner).

The graph is ready, at the level of roads or of lanes. Its edges still need a cost, and length is

not the one a driver cares about.

4. Travel time as the edge cost

> **Key point:** The cost of a road is the time it takes: its length divided by its speed limit. With that cost, the fastest route in the town runs 3300 m by the bypass in 197.1 s, while the shortest, 2500 m through the old town, takes 252.0 s.

Section 3 left the edges of the road graph without a cost. In the room, a move cost its length, and the **weighted graph** (G-2563) of the room's grid gave each edge its length in metres. On roads, the same choice picks the wrong route.

The problem on numbers. From S to T, the straight way through the old town is 2500 m; the way round by the bypass is 3300 m. By length, the old town route is shorter. But the old town's streets allow 30 km/h and the bypass 80 km/h.

The idea. A driver judges a route by the time it takes. Driving a street at its speed limit takes its length divided by that speed, so each edge gets that time as its cost (Rhodes COMP 372; Tübingen L12 slides). A speed in km/h becomes m/s by dividing by 3.6, because 1 km is 1000 m and 1 h is 3600 s. For the bypass from N to M, 1500 m at 80 km/h:

$$80 \text{ km/h} = 80/3.6$$

$$= 22.22 \text{ m/s}$$

$$\text{time} = 1500/22.22$$

$$= 67.5 \text{ s}$$

With:

- ℓ, the length of the street, in m (1500 m from N to M);
- v, its speed limit, in m/s (22.22 m/s);
- c, the cost of its edge, in s (67.5 s).

$$c = \ell / v$$

The standard term. An edge cost equal to the time it takes to drive the edge, here its length divided by its speed limit, is a **travel-time edge cost** (G-2876) (Tübingen L12 slides; Freiburg L3). Figure 6 labels every street of the town with it, and draws both routes.

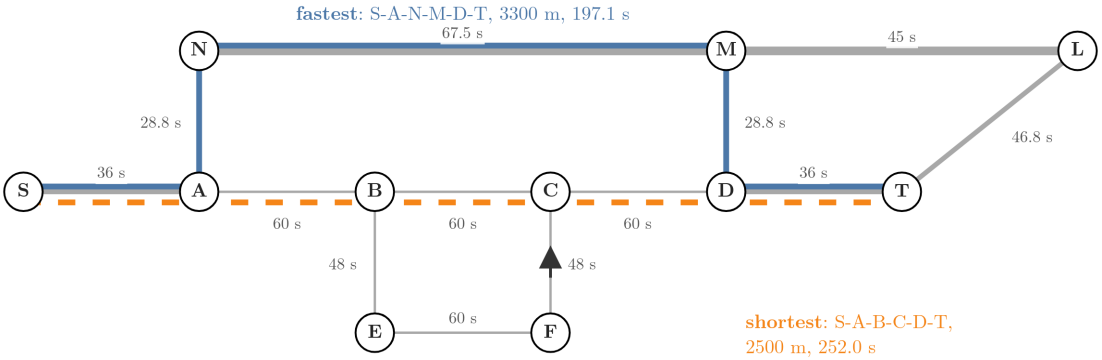


Figure 6: The town with every street labelled by its travel time in s (length divided by speed limit). Shortest by length (orange, dashed): S-A-B-C-D-T, 2500 m, 252.0 s, through the old town. Fastest (blue): S-A-N-M-D-T, 3300 m, 197.1 s, by the bypass. The two routes share S-A and D-T; they are drawn side by side there

The two routes, street by street:

$$\text{old town} = 36 + 60 + 60 + 60 + 36$$

$$= 252.0 \text{ s}$$

$$\text{bypass} = 36 + 28.8 + 67.5 + 28.8 + 36$$

$$= 197.1 \text{ s}$$

The bypass is 800 m longer and 54.9 s faster. Dijkstra's algorithm (G-2565), run with travel-time costs, returns the bypass, since it minimises the sum of the costs whatever they measure (notebook).

On the Chicago trip the same happens at a larger scale: the fastest route, 7047 m, takes 492.8 s, and the shortest, 6717 m, takes 573.1 s (notebook). OpenStreetMap stores a speed limit for only some streets, 313 of the 3946 saved (notebook); OSMnx gives every other street the average limit of the tagged streets of its road type, or of all tagged streets when none of its type is tagged (OSMnx documentation).

Dijkstra finds the fastest route, but it took 1526 of 1628 intersections off its open list on the Chicago trip. A* needs a heuristic, and the heuristics of the room measure metres, not seconds.

5. A heuristic in seconds: the straight line at the top speed

> **Key point:** Divide the straight-line distance to the goal by the highest speed limit on the map: no route can be shorter than the straight line or faster than the top speed, so the guess never overestimates. But most roads are much slower than the top speed, so the guess is loose: in Chicago it averages 0.41 of the true time, and A* still takes off 1229 of the trip's intersections.

Section 4 left A* without a heuristic for travel time. The room's straight-line and octile distances are in metres; the costs are now in seconds, and A* adds the two, so a heuristic in metres cannot be used.

5.1 The bound and why it holds

The idea. The time left from an intersection is at least the time it would take to drive the straight line to the goal at the fastest speed anywhere on the map. For the town, the fastest speed is the bypass's 80 km/h, 22.22 m/s. From S, 2500 m from T in a straight line:

$$h(S) = 2500/22.22$$

$$= 112.5 \text{ s}$$

With:

- u, an intersection;
- $\delta(u)$, the straight-line distance from u to the goal T, in m (2500 m from S);
- $v_{\max}$, the highest speed limit on the map, in m/s (22.22 m/s in the town);
- $h(u)$, the heuristic at u, in s (112.5 s at S).

$$h(u) = \delta(u)/v_{\max}$$

This heuristic is the straight-line distance over the top speed (Freiburg L3; Rhodes COMP 372).

Why it never overestimates. Take the fastest route from S, five streets of 500, 400, 1500, 400 and 500 m. Driving each street at the top speed instead of its own limit can only make it faster, and no route is shorter than the straight line:

$$\text{time} = 36 + 28.8 + 67.5 + 28.8 + 36$$

$$= 197.1$$

$$\geq 3300/22.22$$

$$= 148.5$$

$$\geq 2500/22.22$$

$$= 112.5$$

$$= h(S)$$

The same steps for any route from u to the goal, with:

- ℓ_i, the length of street i of the route, in m;
- v_i, its speed limit, in m/s, never above $v_{\max}$;
- $\sum_i$, the sum over the streets of the route (five terms for the route from S, one per street).

$$\text{time} = \sum_i \ell_i / v_i$$

$$\geq \sum_i \ell_i / v_{\max}$$

$$\geq \delta(u)/v_{\max}$$

$$= h(u)$$

The second line holds because every v_i is at most $v_{\max}$; the third because the streets of a route together are never shorter than the straight line between its ends. So the heuristic is **admissible** (G-2578): it never overestimates, and A* returns the fastest route. The heuristic is also **consistent** (G-2580), for the same reason applied to one edge at a time (Freiburg L3).

Rhodes College's Memphis exercise uses the same bound with 65 miles per hour, the highest speed limit of any Memphis road (Rhodes COMP 372).

5.2 Why the bound is loose

A heuristic that never overestimates is safe; how much work it saves depends on how close it comes to the truth: of two admissible heuristics, the one that is never smaller, which **dominates** (G-2579) the other, takes fewer vertices off the open list. At S it guesses 112.5 s for a true 197.1 s. Figure 7 runs Dijkstra and then A* with this heuristic on the town; watch run 2's frame "take B off".

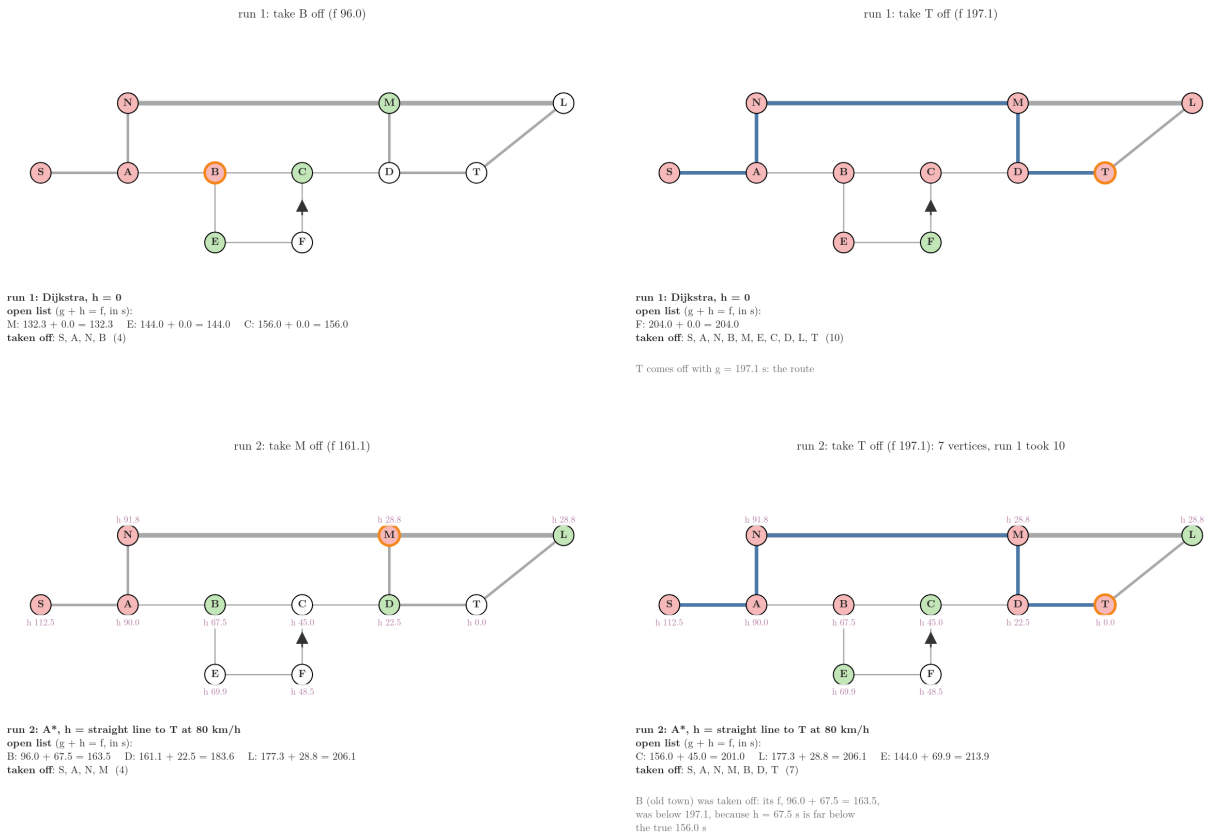


Figure 7: Two searches on the town from S to T with travel-time costs, one vertex taken off the open list per frame. Each frame lists the open list as g + h = f in s, where g is the time from S so far, h the heuristic and f their sum. Run 1: Dijkstra (h = 0) takes off 10 vertices. Run 2: A* with h = the straight line to T at 80 km/h (purple numbers) takes off 7: it skips E, C and L, but still takes B off, because B's f, 96.0 + 67.5 = 163.5 s, is below the route's 197.1 s. Both return S-A-N-M-D-T, 197.1 s

A* takes 7 vertices off its open list against Dijkstra's 10 (notebook). A* still takes off B, in the old town, which the fastest route never uses. B's estimated total cost is its time from S plus the heuristic:

$$f(B) = 96.0 + 67.5$$

$$= 163.5 \text{ s}$$

The 163.5 s is below 197.1 s, so B looks as promising as the bypass. The heuristic at B, 67.5 s, assumes the 1500 m to T are driven at 80 km/h, while the true fastest time from B is 156.0 s, through the 30 km/h old town.

On Chicago the gap is wider. The top speed limit, 88.5 km/h, belongs to the expressways of Figure 4; the median street is 35.3 km/h (notebook). Figure 8 compares the guess with

the truth at every intersection of the map, for the trip's goal: the dots lie far below the line where guess and truth are equal.

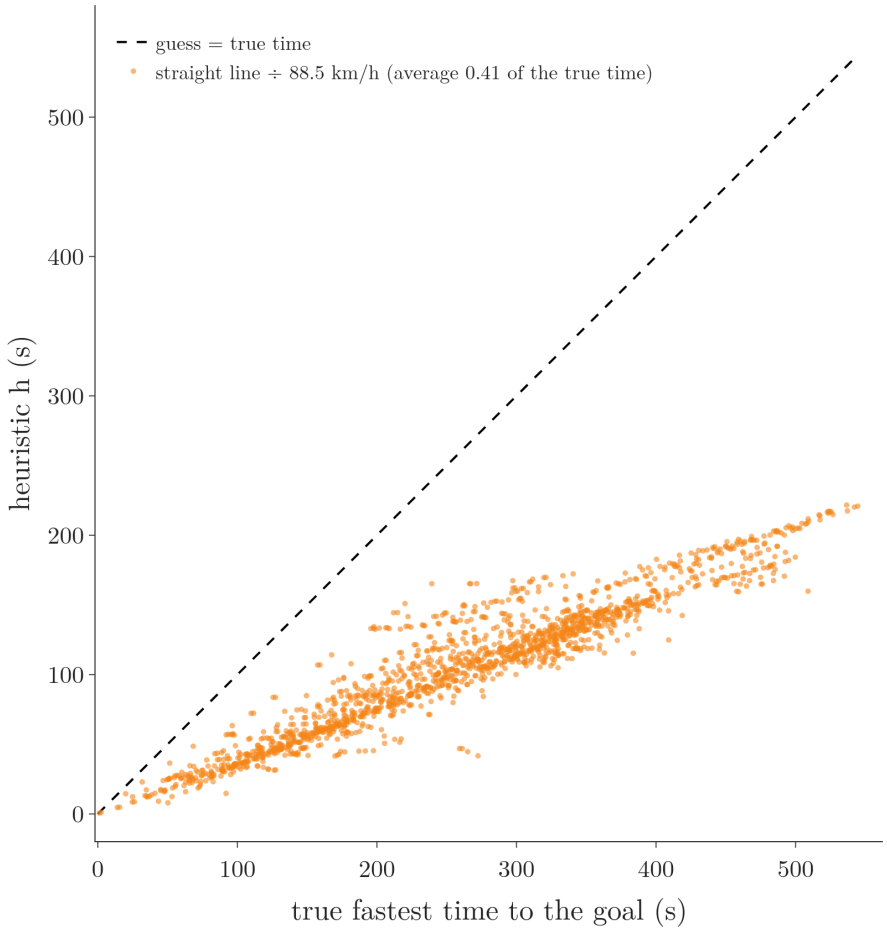


Figure 8: The straight line to the goal at the top speed, 88.5 km/h, against the true fastest time to the goal, at each of the 1628 Chicago intersections (one dot each). Every dot lies under the dashed line guess = true time, so the guess never overestimates; on average it is 0.41 of the true time. At the trip's start the guess is 187.7 s for a true 492.8 s

The average over 200 random trips on the Chicago map shows what the looseness costs (notebook, seed 0):

Cost	Dijkstra takes off	A* with the straight line takes off
length (m), heuristic = straight line	854.5	240.4
travel time (s), heuristic = straight line ÷ 88.5 km/h	861.1	528.1

With length as the cost, the straight line is close to the truth on Chicago's street grid, and A* takes off 28 percent of what Dijkstra does. With travel time, the same line divided by the top speed is loose, and A* takes off 61 percent. A*'s speed depends on having a heuristic that estimates distances well (Strasser and Zeitz 2021), and road networks make the straight line a poor one: Euclidean bounds are "not very good for driving directions" (UW CSE 421 L7). The guess must take the roads into account.

6. Landmarks: lower bounds from stored travel times

> **Key point:** Store, once, the exact travel times between a few landmark vertices and every other vertex; since a detour through the goal is never faster than the fastest way to a landmark, the stored times give a lower bound on the time left that takes the roads into account. With 16 landmarks, A* takes off 108 of the Chicago trip's intersections instead of 1229.

The straight line of Section 5 ignores the roads: at B it guessed 67.5 s where the true time is 156.0 s. A better guess needs the roads, but the exact time left differs for every goal: computing it before each search is too slow, and storing it for every pair of vertices takes too much memory (Red Blob, differential heuristics).

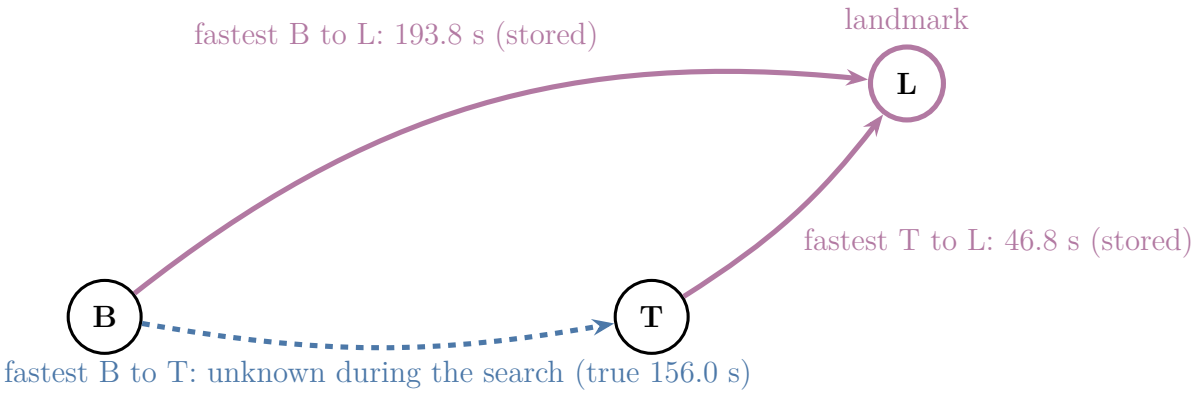
6.1 The triangle inequality on travel times

The idea. Pick one vertex, L at the east edge of the town, and compute the fastest time from every vertex to it, once, with one Dijkstra run backwards from L. Then use it for any goal. From B, one way to reach L is to drive to T first and then on to L. That way can never be faster than the fastest way from B to L (Figure 9). With the stored times, 193.8 s from B to L and 46.8 s from T to L (notebook):

$$\text{time}(B, T) + 46.8 \geq 193.8$$

$$\text{time}(B, T) \geq 193.8 - 46.8$$

$$= 147.0 \text{ s}$$



$$\text{B} \to \text{T} \to \text{L} \text{ is one way to reach L, never faster than the fastest:}$$
$$\text{time(B, T)} + 46.8 \geq 193.8$$
$$\text{time(B, T)} \geq 193.8 - 46.8 = 147.0 \text{ s}$$

Figure 9: The landmark bound at B. The fastest times from B to the landmark L, 193.8 s, and from T to L, 46.8 s, are stored (purple). The fastest time from B to the goal T is what the search does not know (blue, dashed; the true value is 156.0 s). Driving B to T to L is one way of reaching L, so it is never faster than the fastest way: time(B, T) + 46.8 ≥ 193.8, so time(B, T) ≥ 147.0 s

The guess at B rises from 67.5 s to 147.0 s; the truth is 156.0 s. The rule used is the **triangle inequality** (G-2683), a detour through a third point is never shorter than going straight, here for fastest travel times instead of distances (Red Blob, differential heuristics; Freiburg L3).

The formal version. With:

- u, the vertex the search is at (B above);
- t, the goal (T);
- L, a landmark (L);
- $\text{time}(x, y)$, the fastest travel time from x to y, in s;
- $h_L(u)$, the heuristic at u from landmark L, in s (147.0 s at B).

The steps, as above with numbers:

$$\text{time}(u, t) + \text{time}(t, L) \geq \text{time}(u, L)$$

$$\text{time}(u, t) \geq \text{time}(u, L) - \text{time}(t, L)$$

$$h_L(u) = \text{time}(u, L) - \text{time}(t, L)$$

On a directed graph the trip into a landmark and the trip out of it can differ, so each landmark stores both, and a second bound comes from driving L to u to t (Freiburg L3):

$$\text{time}(u, t) \geq \text{time}(L, t) - \text{time}(L, u)$$

The heuristic takes the larger of the two bounds, and 0 if both are negative. Both lines are lower bounds, so the heuristic is admissible. The landmark heuristic is also consistent (Freiburg L3). On the edge from A to N, 28.8 s, the stored times to L are 141.3 s from A and 112.5 s from N:

$$141.3 \leq 28.8 + 112.5$$

$$141.3 - 46.8 \leq 28.8 + 112.5 - 46.8$$

$$94.5 \leq 28.8 + 65.7$$

The same steps for any edge from u to w with travel time c:

$$\text{time}(u, L) \leq c + \text{time}(w, L)$$

$$h_L(u) \leq c + h_L(w)$$

The first line is the triangle inequality again; the second subtracts $\text{time}(t, L)$ from both sides.

The standard terms. A vertex chosen in advance whose travel times to and from every other vertex are stored is a **landmark** (G-2877) in route planning; it is a vertex of the graph, not an object the robot senses, as a landmark (G-2342) is in localisation. A* with the largest landmark bound as its heuristic is **ALT** (G-2878), for A*, landmarks and the triangle inequality (Goldberg and Harrelson 2005). The bound is also called the differential heuristic, because it takes the difference of two stored costs (Red Blob, differential heuristics).

6.2 The search with a landmark

The bound at B is now close to the truth. Figure 10 runs A* with it on the town; watch B through the frames: it goes onto the open list and is never taken off.

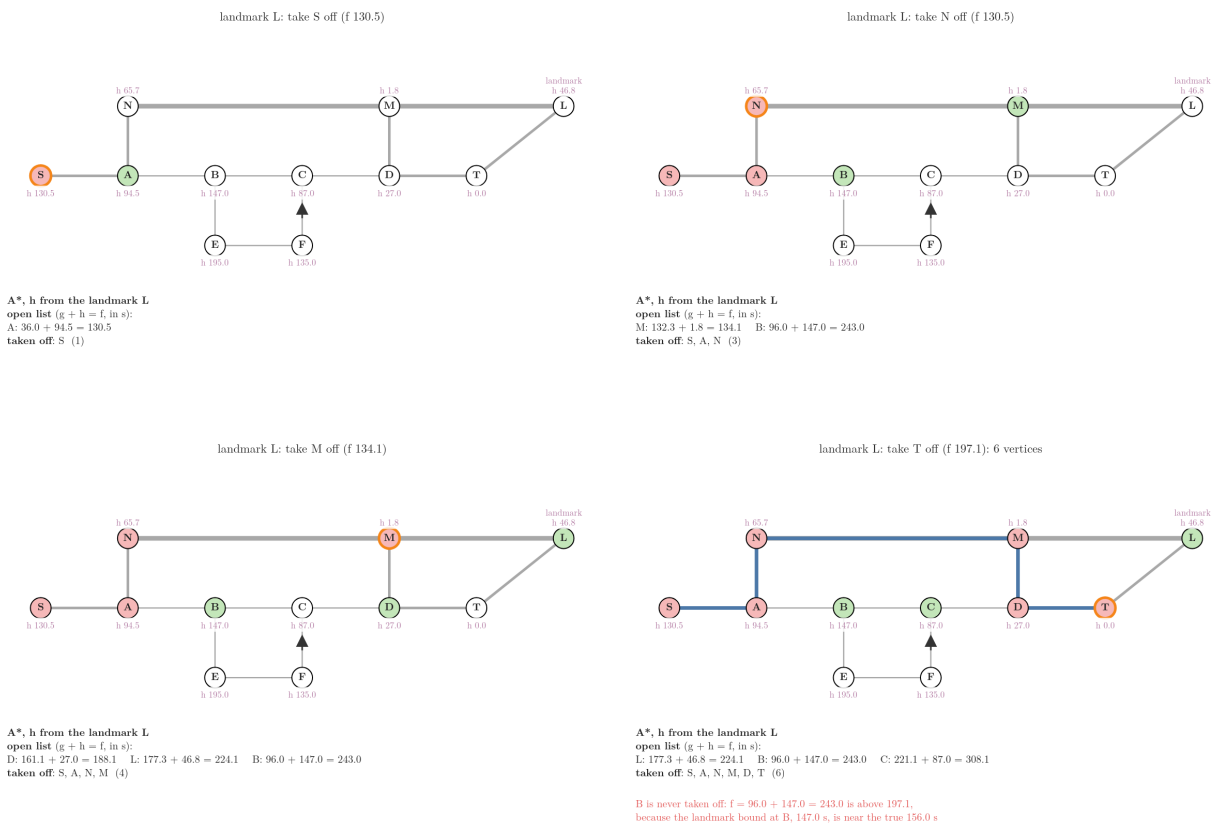


Figure 10: A* on the town from S to T with the landmark heuristic from L (purple numbers: h at each vertex, in s), one vertex taken off the open list per frame; each frame lists the open list as g + h = f in s, where g is the time from S so far, h the heuristic and f their sum. A* takes off S, A, N, M, D and T, 6 vertices, only the fastest route. B, in the old town, stays on the open list: its f = 96.0 + 147.0 = 243.0 s is above the route's 197.1 s

With the landmark, B's estimated total cost is:

$$f(B) = 96.0 + 147.0$$

$$= 243.0 \text{ s}$$

which is above the route's 197.1 s, so A* finishes before B comes off: 6 vertices against 7 with the straight line and 10 with Dijkstra, all with the 197.1 s route (notebook).

One landmark does not help everywhere. At M the bound is only 1.8 s, while the straight line gives 28.8 s and the truth is 64.8 s (notebook): from M the fastest way to L does not pass T, so the detour through T gives a weak bound. A landmark gives a tight bound when it lies behind the goal, as seen from the vertex, or before the vertex, as seen from the goal (Freiburg L3; Goldberg and Harrelson 2005). No single landmark is behind every goal, so ALT keeps several landmarks and takes the largest bound (Red Blob, differential heuristics).

6.3 Many landmarks on a real map

On the Chicago map we choose 16 landmarks by the farthest rule: start anywhere, take the vertex farthest away by travel time, then repeatedly the vertex farthest from all the landmarks chosen so far (Goldberg and Harrelson 2005). Figure 1 (bottom left) shows them as purple diamonds, most near the edges of the map. Each landmark costs two Dijkstra runs, one out of it and one into it, and its times are stored for all vertices:

$$16 \times 1628 \times 2 = 52096 \text{ times}$$

Figure 11 adds the 16-landmark guess to Figure 8: most purple dots sit near the line where guess and truth are equal.

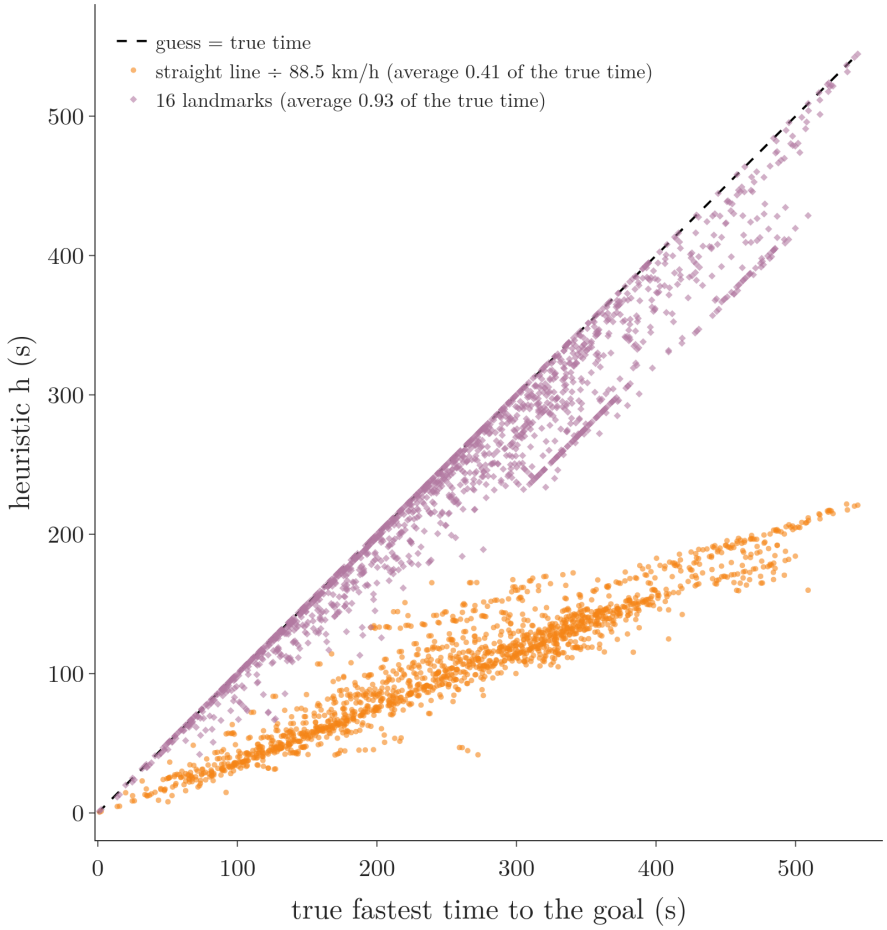


Figure 11: As Figure 8, with the heuristic from 16 landmarks added (purple diamonds): at each Chicago intersection, the largest landmark bound against the true fastest time to the goal. On average it is 0.93 of the true time, against 0.41 for the straight line at the top speed (orange). At the trip's start: 434.5 s for a true 492.8 s, against 187.7 s

The closer guess means less work, by the same dominance rule as on the room's grid. Averaged

over 200 random trips (notebook, seed 0):

Heuristic	Intersections taken off
none (Dijkstra)	861.1
straight line ÷ top speed	528.1
1 landmark	398.2
4 landmarks	115.3
8 landmarks	91.7
16 landmarks	51.6

Even one landmark does better than the straight line, and more landmarks help further, as Goldberg and Harrelson found on road networks (Goldberg and Harrelson 2005). On a road graph of 1.6 million vertices with travel-time costs, A* with landmarks, searching from both ends, took 12 ms where bidirectional (G-2584) Dijkstra took 340 ms (UW CSE 421 L7).

Landmarks keep the search near the route, but the search still follows the route itself, intersection by intersection. A long route passes thousands of vertices, such as the 3,617 of a route from Glasgow to London (mjt CH), and A* takes every one of them off its open list.

7. Contraction hierarchies: shortcuts over unimportant roads

> **Key point:** Rank the vertices from least to most important and remove them in that order, adding a shortcut whenever a removed vertex lay on the only fastest way between two of its neighbours. A query then runs two searches, from the start and from the goal, that only climb to higher ranks and meet at the top, which the shortcuts make enough: 67 + 92 intersections on the Chicago trip.

ALT of Section 6 still takes off its open list every intersection of the route and some around it: 108 on the Chicago trip, whose route has 42, and the route grows with the trip. Long trips need a search that skips most of the small roads.

7.1 Why skip small roads

The idea. Drivers use small streets near the start and the goal, bigger roads farther away, and motorways in between (Freiburg L6). On the town trip, only S and T are on small roads; the middle of the route is the bypass. A search would be faster if, away from the ends, it looked only at important roads.

A rule such as “far from the ends, use only motorways” is fast but can miss the fastest route; navigation devices used rules of that kind for years (Freiburg L6). Contraction hierarchies keep the idea and lose nothing: they add shortcuts so that skipping a vertex never loses a route.

7.2 Contracting a vertex: shortcuts

The idea. Put the vertices in an order, least important first: in the town, the old town (E, F, B, C), then the ends (S, T, L), the junctions (A, D) and the bypass last (N, M). Remove them one at a time. Before removing a vertex, look at every way through it, from a neighbour u in to a neighbour w out:

- if the way through it is the only fastest way from u to w, add a direct edge from u to w with the same time;
- if another way from u to w, avoiding the vertex, is no slower, add nothing.

The standard terms. Removing a vertex while keeping every fastest time between the vertices left is **node contraction** (G-2879) (Geisberger et al. 2008). The direct edge it adds is a **shortcut** (G-2880). The other way is found by a Dijkstra run from u that may not use the vertex being contracted (Freiburg L6; Geisberger et al. 2008). Figure 12 contracts the town.

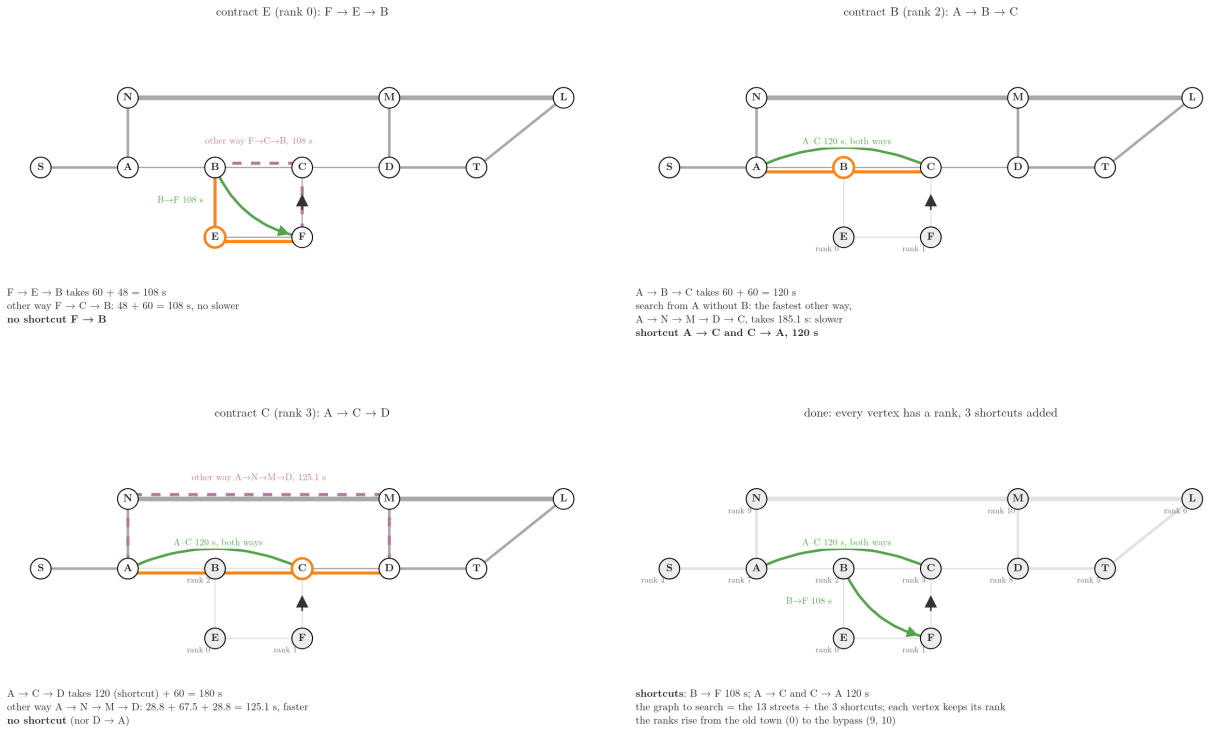


Figure 12: The town contracted one vertex at a time in the order E, F, B, C, S, T, L, A, D, N, M (orange outline: the vertex being contracted; pale: contracted, with its rank). For each way through the vertex (orange), a search that avoids the vertex looks for another way that is no slower (purple, dashed). None: a shortcut (green) with the way's time. Contracting E adds B → F, 108 s; contracting B adds A → C and C → A, 120 s; contracting F, C and T finds other ways that are no slower. The last frame shows the 3 shortcuts and every vertex's rank

Two steps of Figure 12 on numbers. Contracting B (frame "contract B"): the way A → B → C takes

$$60 + 60 = 120 \text{ s}$$

and with B removed (and E, F already gone) the fastest other way, A → N → M → D → C, takes

$$28.8 + 67.5 + 28.8 + 60 = 185.1 \text{ s}$$

which is slower, so the shortcut A → C of 120 s is added (and C → A). Contracting C next (frame "contract C"): the way A → C → D, using that shortcut, takes

$$120 + 60 = 180 \text{ s}$$

and the other way, A → N → M → D, takes

$$28.8 + 67.5 + 28.8 = 125.1 \text{ s}$$

which is faster, so no shortcut. After every vertex has gone, each has a rank, its place in the order, and the graph to search is the original edges plus the shortcuts: the **contraction hierarchy** (G-2882), CH (Geisberger et al. 2008).

Why contraction keeps every fastest time. Each contraction keeps every fastest time between the vertices that remain: a fastest route through the removed vertex either has a shortcut of the same time or another way that is no slower (Freiburg L7). In the town, after B is gone, A to C still takes 120 s, by the shortcut. Applying this once per vertex, the times in the final graph equal those of the original (Freiburg L7).

The order decides the speed, not the answer. Any order gives correct routes; a good order adds few shortcuts and gives short searches (Freiburg L6; mjt CH). A common order prefers vertices whose contraction adds fewer edges than it removes, and spreads the contractions evenly over the map by counting neighbours already contracted (Geisberger et al. 2008). On a road graph of Western Europe, 18,029,721 vertices, the first rule alone gave queries that took 1791 vertices off; adding the count of contracted neighbours cut that to 403 (Geisberger et al. 2008). The Chicago hierarchy uses those two rules and adds 5509 shortcuts to its 3857 edges (notebook).

7.3 The query: two searches that only climb

The idea. Search from the start and, backwards, from the goal, as in bidirectional search (G-2584), but let each search follow only edges that lead to a higher rank. Each climbs towards the important roads, and they meet near the top (Geisberger et al. 2008). Figure 13 runs it on the town; the bottom of each frame redraws the vertices at the height of their rank.

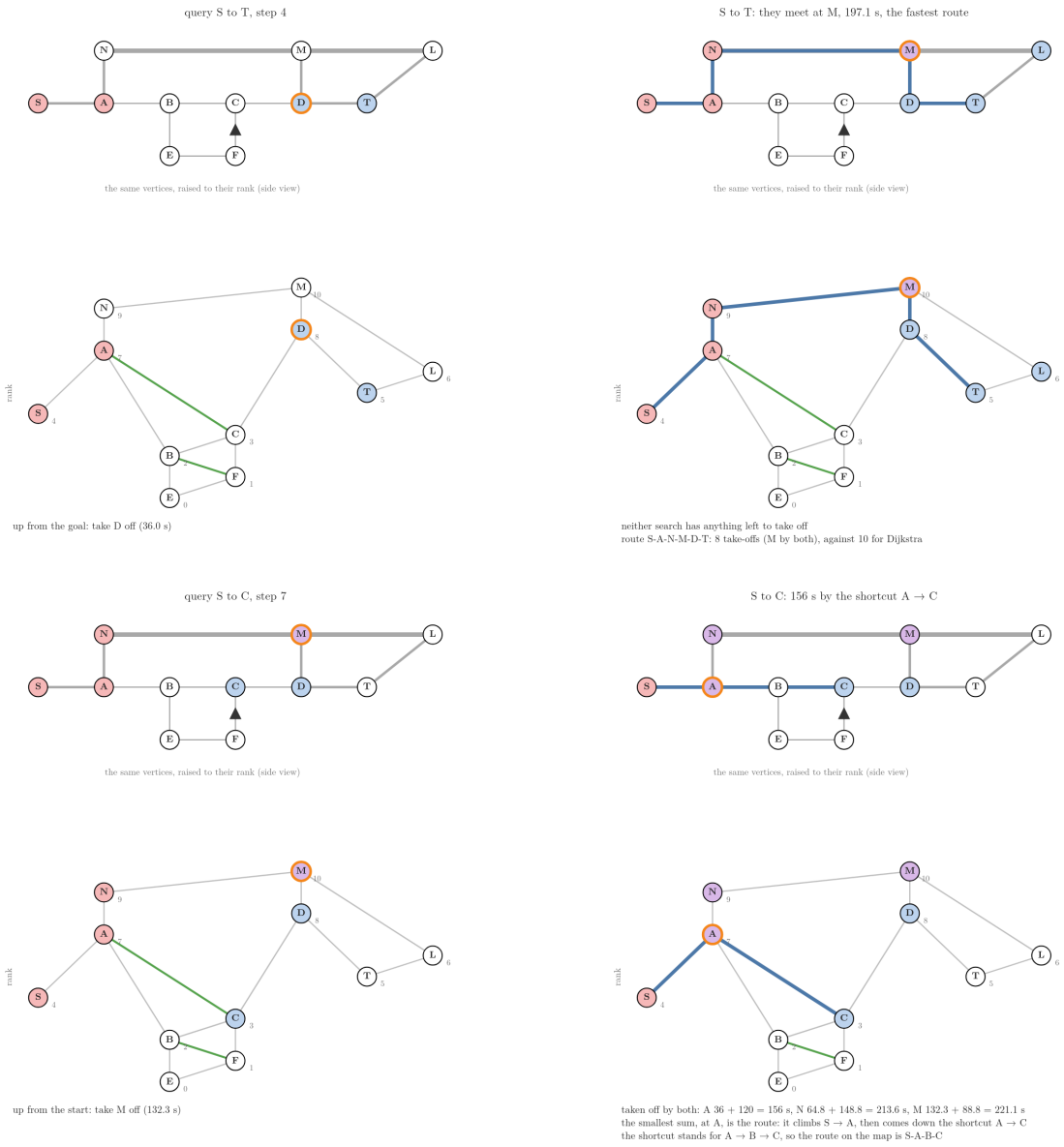


Figure 13: Queries on the contracted town. Top: the town; bottom: the same vertices raised to their rank (0 for E at the bottom, 10 for M at the top), with the edges and shortcuts (green) between them. The search from the start (red) and the search back from the goal (blue) take one vertex off in turn and follow only edges that climb; purple: taken off by both. S to T: they meet at M, 132.3 + 64.8 = 197.1 s, after 8 take-offs. S to C: the cheapest vertex taken off by both is A, 36 + 120 = 156 s, by the shortcut A → C, which unpacks to A, B, C on the map

On the town trip, the search from S climbs S, A, N, M, and the search back from T climbs T, D, L, M. M is taken off by both:

$$132.3 + 64.8 = 197.1 \text{ s}$$

the fastest time, after 8 take-offs, against Dijkstra's 10. Each search stops once the smallest cost on its open list is at least the best route found so far, as no later vertex can give a faster one (Geisberger et al. 2008). The answer is the smallest sum over all vertices taken off by both (Freiburg L6), as in bidirectional search: for S to C three vertices qualify, A with 156 s, N with 213.6 s and M with 221.1 s, and A gives the smallest (Figure 13, last frame).

Why climbing is enough. Follow the ranks along the fastest town route S-A-N-M-D-T:

Vertex	S	A	N	M	D	T
Rank	4	7	9	10	8	5

The ranks rise to M, the most important vertex on the route, and then fall. So the route is a climb from S to M, found by the search from S, and a climb from T to M, found by the search from T. A route whose ranks dip in the middle is also covered: the dip's vertex was contracted before its two neighbours, and a shortcut, or another way of the same time, bridges it. On S to C, B (rank 2) lies between A (rank 7) and C (rank 3), and the shortcut A $\rightarrow$ C bridges it; unpacking the shortcut gives back A, B, C (Freiburg L7; Geisberger et al. 2008).

On real maps. On the Chicago trip the two searches take off 67 and 92 intersections, 159 in all, and over 200 random trips 90.0 on average, against 861.1 for Dijkstra and 51.6 for 16 landmarks (notebook). On a city map, the hierarchy and the landmarks are close. The hierarchy is most useful on large maps: on Western Europe's 18 million vertices, a query took 403 vertices off and ran in 183 µs (Geisberger et al. 2008), and on the United Kingdom's 3,478,051 vertices a query visited 3968 + 3113 of them (mjt CH).

7.4 CH-Potentials: the hierarchy as a heuristic

A contraction hierarchy is built for one set of costs. Traffic changes the times, and forbidden turns or other rules change which routes are allowed; A* handles such changes easily, because only its costs and moves change, but it is only as fast as its heuristic (Strasser and Zeitz 2021). **CH-Potentials** (G-2883) take the heuristic for A* from a contraction hierarchy: the hierarchy's fast queries give A* tight estimates of the time left, while A* keeps its flexibility (Strasser and Zeitz 2021). Here the heuristic is built from the road network's own hierarchy.

From no heuristic to the straight line to landmarks, the A* loop never changed; only the heuristic did, and with it the work, from 1526 intersections to 108. CH-Potentials take the same step once more: the hierarchy itself becomes the heuristic.

8. Summary

Method	Precomputed	Chicago trip, taken off	200 trips, average	Why use it
Dijkstra (heuristic 0)	nothing	1526	861.1	no heuristic needed
A*, straight line ÷ top speed	nothing	1229	528.1	no precomputation, but loose when most roads are slower than the top speed
A*, 16 landmarks (ALT)	2 Dijkstra runs per landmark, 52096 times	108	51.6	tight bounds from real travel times
contraction hierarchy	an order and 5509 shortcuts	67 + 92 = 159	90.0	searches stay small on very large maps

All four return the fastest route, 492.8 s on the Chicago trip, because the three heuristics never overestimate and the shortcuts keep every fastest time.

- The Chicago trip of the opening needs no 96 million cells: on its road graph of 1628 intersections the fastest route, 492.8 s, is found by taking 1526 intersections off the open list with no heuristic, 1229 with the straight line at the top speed and 108 with 16 landmarks, because the A* loop is the same and only the heuristic, the estimate of the time left, changes; a contraction hierarchy, which adds shortcuts before the search, takes 159.
- **Route planning** (G-2870) is the top planning level, so it can work on a map of roads and lanes and leave the motion to the levels below.
- A **road graph** (G-2871) has intersections as vertices and road segments as edges; it is a **directed graph** (G-2873), because a one-way street is an edge that goes one way only. A **route graph** (G-2872) does the same for a robot's waypoints.
- A **lane graph** (G-2874) has one vertex per **lanelet** (G-2875), so the route can say which lane to be in before a turn, with a penalty on each lane change.
- A **travel-time edge cost** (G-2876) is length divided by speed limit, so the fastest route can be longer: 3300 m and 197.1 s by the town's bypass against 2500 m and 252.0 s through the old town.
- The straight line divided by the top speed never overestimates, because no route is shorter than the straight line or faster than the top speed; it is loose, 0.41 of the truth on average in Chicago, because most streets are much slower.
- A **landmark** (G-2877) stores fastest times to and from every vertex, so the triangle inequality gives a lower bound that follows the roads; **ALT** (G-2878) takes the largest over several landmarks, 0.93 of the truth on average with 16.

- **Node contraction** (G-2879) removes the least important vertex and adds a **shortcut** (G-2880) only when a search that avoids that vertex finds no other way that is as fast, so every fastest time is kept.
- A **contraction hierarchy** (G-2882) answers a query with two searches that only climb, because every fastest route rises to its most important vertex and falls again; on large maps the searches stay at a few hundred vertices.
- **CH-Potentials** (G-2883) turn a contraction hierarchy into a heuristic for A*, so A* stays flexible for traffic and turn rules and still searches little.

The robot can now pick the fastest roads and lanes across a city, quickly. Still missing: a route is a list of roads, and the motion planner below it must still turn each piece into a motion the robot can drive, with its speeds and turns over time; trajectory optimisation finds that motion by improving it step by step.

9. Sources

Built from

- Tübingen Machine Learning, "Self-Driving Cars - Lecture 12.1 (Decision Making and Planning: Introduction)", YouTube, https://www.youtube.com/watch?v=VAhBWF3PRM8, and "Self-Driving Cars - Lecture 12.2 (Decision Making and Planning: Route Planning)", YouTube, https://www.youtube.com/watch?v=1T0trI6yDQc, with the lecture's slides (University of Tübingen), https://drive.google.com/file/d/120i8b-p8CfzUofzZQgxDFg4odugHURIb/view (Tübingen L12 slides)
- Veritasium, "Google Maps is unreasonably fast. Let me explain", YouTube, https://www.youtube.com/watch?v=kS-CGkiPetQ
- Lanelet2 (FZI and KIT), "Lanelet2 routing" documentation, https://github.com/fzi-forschungszentrum-informatik/Lanelet2/blob/master/lanelet2_routing/README.md (Lanelet2 routing)
- Autoware, "autoware_mission_planner" documentation, https://index.ros.org/p/autoware_mission_planner/ (Autoware mission planner)
- Open Navigation, Nav2 "Route Server" (nav2_route) documentation, https://github.com/ros-navigation/navigation2/blob/main/nav2_route/README.md (Nav2 Route Server)
- Bast, H. (University of Freiburg), *Efficient Route Planning*, SS 2012, lecture 3 slides "A*, Landmarks, Set Dijkstra", https://ad-teaching.informatik.uni-freiburg.de/route-planning-ss2012/lecture-3.pdf, and lectures 6 and 7 "Contraction Hierarchies", https://ad-wiki.informatik.uni-freiburg.de/teaching/EfficientRoutePlanningSS2012 (Freiburg L3, L6, L7)
- Patel, A. (Red Blob Games), "Improving heuristics for A* search" (differential heuristics), https://www.redblobgames.com/pathfinding/heuristics/differential.html (Red Blob, differential heuristics)
- mjt.me.uk, "Contraction Hierarchies path finding algorithm, illustrated using three.js" (2015), https://www.mjt.me.uk/posts/contraction-hierarchies/ (mjt CH)

- Lee, Y. T. (University of Washington), CSE 421 (winter 2022), lecture 7 slides, route-planning images from A. V. Goldberg, https://courses.cs.washington.edu/courses/cse421/22wi/lecture/lecture-7.pdf (UW CSE 421 L7)
- Kirlin, P. (Rhodes College), COMP 372 Artificial Intelligence, "Project 1: Navigating Memphis", https://cs.rhodes.edu/~kirlinp/courses/ai/f18/projects/proj1 (Rhodes COMP 372)
- Goldberg, A. V. and Harrelson, C. (2005). "Computing the shortest path: A* search meets graph theory". *Proceedings of the 16th ACM-SIAM Symposium on Discrete Algorithms (SODA)*, 156–165 (Goldberg and Harrelson 2005)
- Geisberger, R., Sanders, P., Schultes, D. and Delling, D. (2008). "Contraction hierarchies: faster and simpler hierarchical routing in road networks". *Experimental Algorithms (WEA 2008)*, Lecture Notes in Computer Science 5038, 319–333 (Geisberger et al. 2008)
- Strasser, B. and Zeitz, T. (2021). "A fast and tight heuristic for A* in road networks". *Symposium on Experimental Algorithms (SEA 2021)*, https://arxiv.org/abs/1910.12526 (Strasser and Zeitz 2021)

Other references

- Paden, B., Čáp, M., Yong, S. Z., Yershov, D. and Frazzoli, E. (2016). "A survey of motion planning and control techniques for self-driving urban vehicles". *IEEE Transactions on Intelligent Vehicles* 1(1), 33–55: the planning levels of Figure 2, as drawn on the Tübingen L12 slides (Paden et al. 2016)
- Boeing, G. (2025). "Modeling and analyzing urban networks and amenities with OSMnx". *Geographical Analysis* 57(4), 567–577, https://doi.org/10.1111/gean.70009, and the OSMnx 2.1.1 documentation of `add_edge_speeds`, https://osmnx.readthedocs.io: the Chicago graph, its speed limits and travel times (Boeing 2025; OSMnx documentation)
- Map data © OpenStreetMap contributors, available under the Open Database License, https://www.openstreetmap.org/copyright: the Chicago streets of Figures 1, 4, 8 and 11

10. Key terms

Terms taught in this Note come first; linked terms are recaps, taught in the Note the link opens.

Term	Meaning
Route planning (mission planning) (G-2870)	Finding the best way from where a vehicle is to its destination on a map of the road network, by travel time or by distance, such as the 492.8 s route across the Chicago map; the top level of planning, which picks the roads and lanes and leaves the motion to the levels below.

Term	Meaning
Route graph (G-2872)	A fixed graph of points a robot may travel between, such as corridor junctions and doors, made by hand or by a program from the map; a robot plans its long trips on it and leaves the last metres to a free-space planner.
Road graph (G-2871)	A graph of a road network whose vertices are intersections and whose edges are the road segments between them, such as 1628 Chicago intersections joined by 3857 edges; it lets a route planner search roads instead of every point of the map.
Directed graph (G-2873)	A graph in which each edge goes one way, from one vertex to another, such as a one-way street from F to C; a two-way street is two edges, so road networks with one-way streets are directed graphs.
Lanelet (G-2875)	One short piece of one lane, given by its left and right borders, as used by the Lanelet2 map format; the vertex of a lane graph.
Lane graph (G-2874)	A graph whose vertices are lane segments (lanelets), joined to the next segment in the same lane and to the neighbouring lanes a vehicle may change into; it lets a route say which lane to be in, for example before a left turn.
Travel-time edge cost (G-2876)	The cost of a road edge given as the time to drive it, its length divided by its speed limit, such as 1500 m at 80 km/h = 67.5 s; with it a search finds the fastest route, which can be longer than the shortest.
Landmark (route planning) (G-2877)	A vertex chosen in advance whose fastest travel times to and from every other vertex are computed once and stored; by the triangle inequality the stored times give a lower bound on the time between any two vertices. Not the sensed landmark of localisation.

Term	Meaning
ALT (A*, landmarks, triangle inequality; differential heuristic) (G-2878)	A* whose heuristic is the largest lower bound from a few landmarks, time(u, L) − time(t, L) and time(L, t) − time(L, u); it never overestimates and follows the roads, so on the Chicago map 16 landmarks cut a trip from 1229 vertices taken off to 108.
Node contraction (G-2879)	Removing a vertex from a graph while keeping every fastest time between the vertices left, by adding a shortcut for each way through it that nothing else matches; the step that builds a contraction hierarchy.
Shortcut (contraction hierarchies) (G-2880)	A direct edge from u to w added when a contracted vertex v lay on the only fastest way u → v → w, with that way's time, such as A → C, 120 s, for A → B → C; it lets a search skip v and is unpacked back into the original edges.
Contraction hierarchy (CH) (G-2882)	A road graph whose vertices have been contracted one by one in order of importance, so each has a rank, plus the shortcuts added; a query runs two searches, from the start and back from the goal, that only climb to higher ranks and meet near the top, so on large maps it looks at a few hundred vertices.
CH-Potentials (G-2883)	A heuristic for A* taken from a contraction hierarchy, which estimates the time left tightly; it keeps A*'s flexibility for traffic and turn rules while using the hierarchy's speed.
A* search (G-2577)	Dijkstra's algorithm with the open list sorted by $f = g + h$, the cost so far plus a heuristic guess of the cost left; with a consistent heuristic it returns the shortest route while expanding far fewer states (9 cells against 54 on the room grid).
Frontier (open list) (G-2547)	The vertices a search has reached but not yet expanded, waiting to be taken off one at a time; which one comes off next is what makes one search differ from another.
Heuristic h (G-2570)	A cheap guess of the cost still to go from a state to the goal, such as 3.121 m from the start cell to the door; a search uses it to prefer states that look nearer the goal.

Term	Meaning
Graph (G-2540)	A set of vertices (points) joined by edges (lines), such as the room's 76 free cells joined by 114 moves; it keeps only which places connect directly, so one search method works for maps, puzzles and any problem of states and moves.
Vertex (graph) (G-2541)	One point of a graph, such as one free cell of the room or one waypoint; the places a search moves between.
Edge (graph) (G-2542)	A link between two vertices of a graph, such as one 0.5 m move between two free cells that share a side; it says the search can go directly from one vertex to the other.
Weighted graph (edge cost) (G-2563)	A graph in which every edge carries a number, its cost, such as a move's length in metres (0.5 m straight, 0.707 m diagonal); the cost of a path is the sum of its edge costs, so the cheapest route need not have the fewest edges.
Dijkstra's algorithm (G-2565)	A search that always settles the vertex with the smallest distance from the start found so far, which is then final, and offers each of its neighbours the route through it, keeping the shorter; with every edge cost at least 0 it finds the shortest route to every vertex (3.414 m to the door on the room grid with diagonals).
Admissible heuristic (G-2578)	A heuristic that is never more than the true cost left from any state, $h \leq h^*$; it makes f a lower bound on every route through a state, so the first route A* takes off the open list is the shortest.
Consistent heuristic (monotone heuristic) (G-2580)	A heuristic that never falls by more than a move's cost, $h(n) \leq c(n, n') + h(n')$, with 0 at the goal (a triangle inequality); it keeps f from decreasing along a route, so A* with a closed list expands every state with its shortest g.
Dominance (of heuristics) (G-2579)	One admissible heuristic dominates another when it is at least as large at every state; being closer to the true cost, it makes A* expand fewer states (the octile distance dominates the straight line).

Term	Meaning
Triangle inequality (G-2683)	The rule that a detour through a third point is never shorter than going straight, $d(p, q) \leq d(p, r) + d(r, q)$; it is what lets a nearest-neighbour search skip a region farther than its best point.
Landmark (G-2342)	An object the robot can detect reliably and tell apart from the others, such as a coloured pole or a door; its known position in the map lets readings of it fix the robot's pose.
Bidirectional search (G-2584)	A forward search from the start and a backward search from the goal run in turn until some state has been expanded by both; the route is the smallest sum of the two costs over all states both reached, since the meeting state can be off the shortest route.